



長安大學

数字逻辑实验课 实验报告

课程名称： 数字逻辑
实验名称： 时序逻辑电路设计
专业名称： 计算机科学与技术/卓越工程师
小组成员：
指导教师： 马峻岩

一、实验目的

1. 学习并掌握运用 Verilog 语言设计时序逻辑电路的原理和方法。
2. 掌握分频电路的设计方法；
3. 基于行为级描述实现 8 位移位寄存器；
4. 掌握 7 段数码管的译码及动态显示原理；
5. 基于行为级描述实现 4 位的十进制计数器；
6. 设计一个具有输入、输出、计时等功能的小型数字系统；

主要仪器和设备：计算机，Basy3 开发板。

二、实验要求

1. Verilog 实现 8 位移位寄存器

(1) 功能要求

表 2-1 移位寄存器真值表

输 入							输出
清零	控制信号		串行输入		时钟	并行输入	Q7~Q0
nclr	S1	S0	Dsr	Dsl	clk	D7~D0	
L	×	×	×	×	×	×	L
H	L	L	×	×	↑	×	Q7~Q0
H	L	H	L	×	↑	×	L Q7~Q1
H	L	H	H	×	↑	×	H Q7~Q1
H	H	L	×	L	↑	×	Q6~Q0 L
H	H	L	×	H	↑	×	Q6~Q0 H
H	H	H	×	×	↑	D7~D0	D7~D0

① 移位寄存器功能如表 2-1 所示，寄存器为上升沿触发。

② Basy3 的外部时钟信号为 100MHz，将时钟信号分频至 1 Hz 后驱动移位寄存器时钟。

③ D7~D0 使用 sw7~sw0；S1 和 S0 为输入控制信号，使用 sw9 和 sw8；Dsr 串行右移输入端，使用 sw11；Dsl 串行左移输入端，使用 sw10；nclr 低电平有效的异步清零信号，使用 sw12；输出的 Q7~Q0 使用 LED7 到 LED0 显示寄存器数据。

④ 寄存器功能由同步的控制信号 s1 和 s0 决定，包括保持、右移、左移和置数。

(2) 其他要求

-
- ① 要求撰写仿真程序，对代码进行仿真测试。
 - ② 将仿真后的 Verilog 代码进行综合与实现，并下载到 Basys3 上验证。

2. Verilog 实现 4 位的十进制计数器

(1) 功能要求

- ① 编写 4 位的十进制计数器，并将计数值显示在 4 位 7 段数码管上。
- ② 计数器驱动时钟频率为 100 Hz。
- ③ sw0 为高电平有效的同步清零信号，sw1 为高电平有效的同步计数使能信号。

(2) 其他要求

- ① 要求撰写仿真程序，对代码进行仿真测试。

3. Verilog 实现秒表

(1) 功能要求

- ① 4 位 LED 数码管 M.SS.D，其中最低位 D 代表 0.1 秒，范围是 0 到 9；SS 代表秒，范围是 00 到 59；M 代表分钟，范围是 0 到 9。
- ② 使用 2ⁿ 分频将数码管扫描频率设定在 1000 Hz 左右。
- ③ sw0 为高电平有效的同步清零信号，sw1 为高电平有效的同步计数使能信号。

(2) 其他要求

- ① 源文件 div.v 实现分频，display.v 实现数码管显示功能，timer.v 为计时功能，timer_tb.v 为仿真文件。
- ② 要求撰写仿真程序，对代码进行仿真测试。
- ③ 将仿真后的 Verilog 代码进行综合与实现，并下载到 Basys3 上验证。

三、相关外设接口及原理

1. Basys3 开发板拨码开关原理图及简介

拨码开关的电路如图 3-1 所示。当开关打到下档时，FPGA 对应引脚输入为低电平；当开关打到上档时，FPGA 对应引脚输入为低电平。

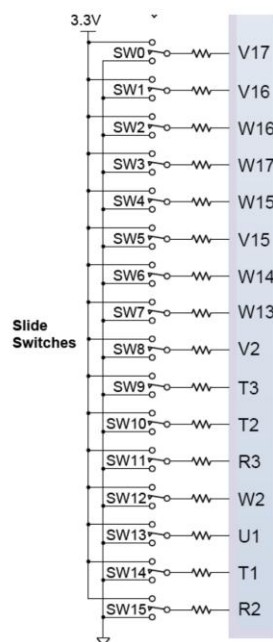


图 3-1 板拨码开关原理图

2. Basys3 开发板数码管电路原理图及简介

数码管显示部分的电路如图 3-2 所示。这里是四位带小数点的七段共阳数码管，当相应的输出脚为低电平时，该段位的 LED 点亮。位选位也是低电平选通。图 3-3 位数码管的十六进制数表示。表 3-1 为共阳极数码管十六进制显示真值表。进行显示时可以通过

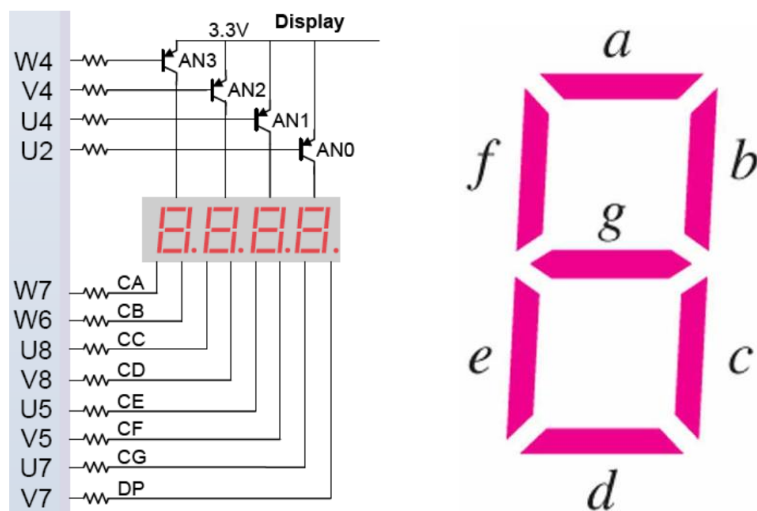


图 3-2 四位数码管电路原理图



图 3-3 数码管十六进制数表示

表 3-1 共阳极数码管十六进制显示真值表

X	a	b	c	d	e	f	g
0	0	0	0	0	0	0	1
1	1	0	0	1	1	1	1
2	0	0	1	0	0	1	0
3	0	0	0	0	1	1	0
4	1	0	0	1	1	0	0
5	0	1	0	0	1	0	0
6	0	1	0	0	0	0	0
7	0	0	0	1	1	1	1
8	0	0	0	0	0	0	0
9	0	0	0	0	1	0	0
A	0	0	0	1	0	0	0
B	1	1	0	0	0	0	0
C	0	1	1	0	0	0	1
D	1	0	0	0	0	1	0
E	0	1	1	0	0	0	0
F	0	1	1	1	0	0	0

3. 数码管扫描显示原理

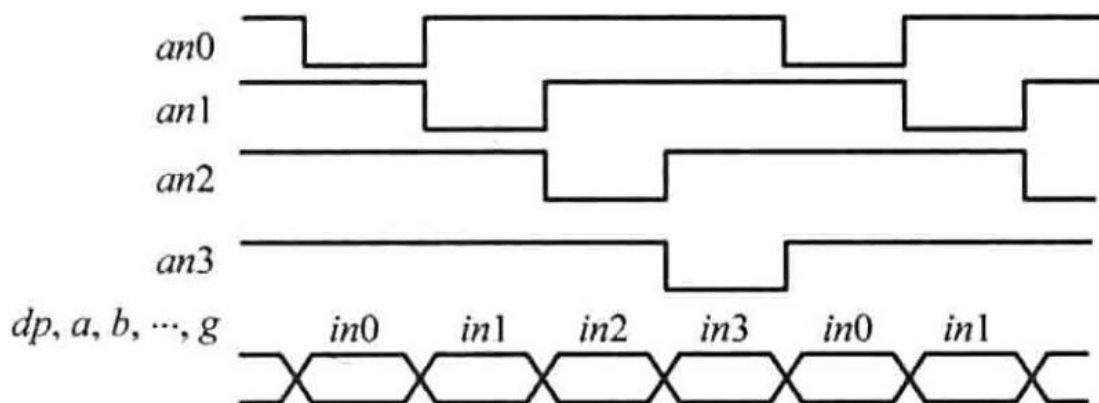


图 3-4 数码管扫描显示电路时序图

图 3-4 给出了四位数码管扫描显示电路时序图，这种控制方式采用分时复用的模

式循环轮流点亮数码管。在同一时间只会点亮一个数码管，点亮的同时公共断码信号输入端 a~g 和 dp 送入对应位需要显示的内容。分时复用的扫描显示利用了人眼的视觉暂留特性，如果扫描速度足够快，人眼就无法区分 LED 的闪烁。然而，如果数码管扫描频率过快会超过数码管开关切换的响应速度，也会产生显示的问题。扫描频率可选 1000Hz 左右。

4. 2ⁿ 分频原理

2ⁿ 分频电路是偶分频电路的一种。为了实现 2ⁿ 分频，需要定义一个长度为 n-1 位的寄存器 clkdiv[n-1:0]，并用输入频率驱动对 clkdiv[n-1:0]进行自增计数，则 clkdiv[n-1]位对应输出为 2ⁿ 分频。除了的到 2 的 n 次幂分频，还可以从 clkdiv 其他位得到 2 的 n-1 次幂、n-2 次幂、.....、1 次幂分频，表 3-2 给出 2ⁿ 分频在 100MHz 输入下的分频表。

表 3-2 2ⁿ 分频在 100MHz 输入下的分频表

表达式	2 ⁿ 分频	频率（MHz）
clkdiv[0]	2 ¹ 分频	100 / 2 = 50
clkdiv[1]	2 ² 分频	100 / 4 = 25
clkdiv[2]	2 ³ 分频	100 / 8 = 12.5
clkdiv[3]	2 ⁴ 分频	100 / 16 = 6.15
clkdiv[4]	2 ⁵ 分频	100 / 32 = 3.125
clkdiv[5]	2 ⁶ 分频	100 / 64 = 1.5625
clkdiv[6]	2 ⁷ 分频	100 / 128 = 0.78125
clkdiv[7]	2 ⁸ 分频	100 / 256 = 0.390625
clkdiv[8]	2 ⁹ 分频	100 / 512 = 0.195313
clkdiv[9]	2 ¹⁰ 分频	100 / 1024 = 0.097656

5. 时序逻辑电路 Basys3 开发板相关约束设置

● 外部时钟信号约束文件

Basys3 的外部时钟信号为 100MHz，通过 W5 引脚连接至 FPGA，使用时需要将约束文件 Clock Signal 段的时钟约束注释删除，示例代码如下。然后在顶层模块将 clk 信号作为 input 接入。

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

- 外部输入信号作为边沿敏感信号的约束文件设置

如果在 always 边沿触发的敏感列表中有 btn, sw 等外部输入, 如 always @ (posedge clk, posedge sw[0]), 需要在约束文增加一行如下代码。注意, 该约束应放在约束文件的最后, 同时注意约束文件格式, 如 implementation 时遇到错误, 把每行约束全部顶格写, 不要有空格。

```
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets { sw [0] }];
```

四、实验步骤及关键代码

1. Verilog 实现 8 位移位寄存器

(1) 主要模块代码

```
1 module shift_register(clrn, s1, s0, Dsr, Dsl, D, clk, Q);
2   input clrn, s1, s0, Dsr, Dsl;
3   input [7:0] D;
4   input clk;
5   output reg [7:0] Q;
6
7   integer k;
8
9   always @(posedge clk, negedge clrn)
10  begin
11    if (!clrn)
12    begin
13      Q <= 8'b00_000_000;
14    end
15    else
16    begin
17      case ({s1, s0})
18        //保持
19        2'b00: Q <= Q;
20
21        //右移
22        2'b01:
23        begin
24          for (k = 0; k <= 6; k = k + 1)
25            Q[k] <= Q[k + 1];
26          Q[7] <= Dsr;
27        end
28
29        //左移
30        2'b10:
31        begin
32          for (k = 7; k >= 1; k = k - 1)
33            Q[k] <= Q[k - 1];
34          Q[0] <= Dsl;
35        end
36
37        //置位
38        2'b11: Q <= D;
39      endcase
40    end
41  end
42
43 endmodule
44
```

```

1 module clock_divide(clk, resetn, clk_div);
2     input clk;           //系统时钟
3     input resetn;        //复位信号
4     output reg clk_div;  //分频时钟
5
6     //parameter n = 26'd50_000_000;
7     reg [25:0] cnt = 0; //计数器
8
9     always @(posedge clk, negedge resetn)
10    begin
11        if (!resetn)      //复位
12        begin
13            cnt <= 0;
14            clk_div <= 0;
15        end
16        else
17        begin
18            if (cnt == 49_999_999) //计数器达到最大值
19            begin
20                clk_div <= ~clk_div;
21                cnt <= 0;
22            end
23            else
24                cnt <= cnt + 1;
25        end
26    end
27
28 endmodule

```

```

1 module shift_register_top(
2     input clk,
3     input wire [12:0] sw,
4     output wire [7:0] led
5 );
6
7     wire clk_div;
8
9     clock_divide dv_clk(.clk(clk), .resetn(sw[12]), .clk_div(clk_div));
10    shift_register sh_r(.clrn(sw[12]), .s1(sw[9]), .s0(sw[8]), .Dsr(sw[11]), .Dsl(sw[10]),
11        .D(sw[7:0]), .clk(clk_div), .Q(led[7:0]));
12 endmodule

```

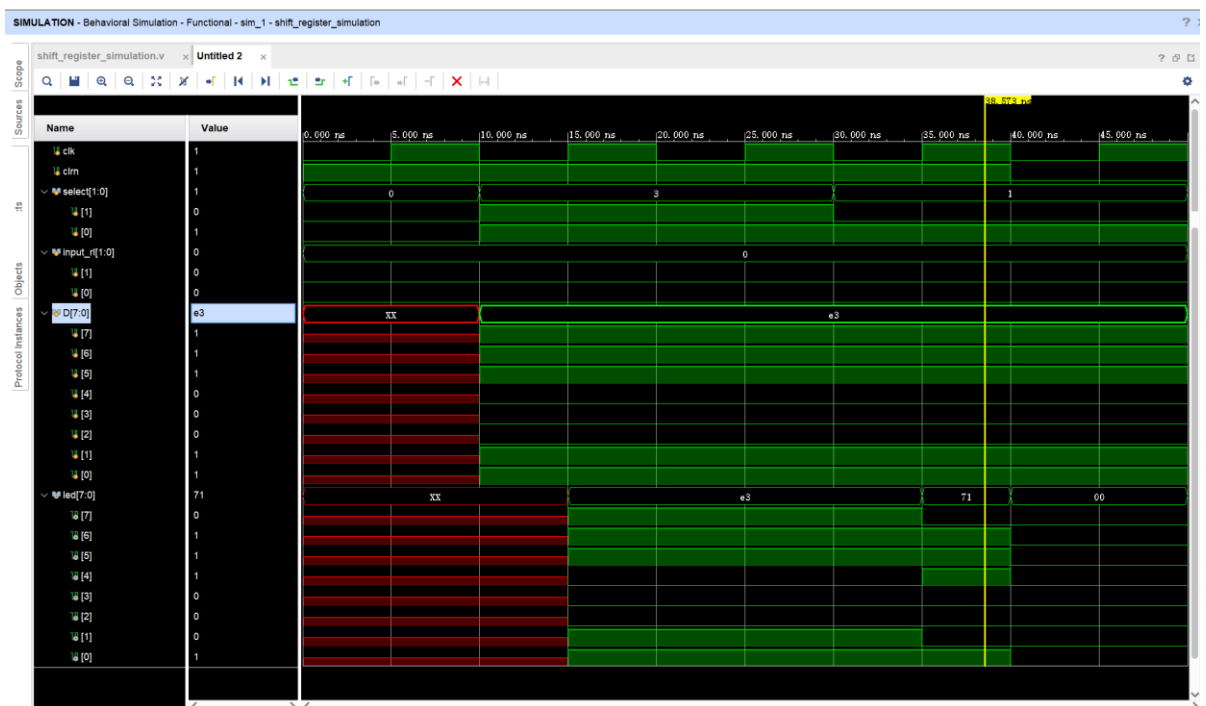
(2) 仿真测试代码


```

1 module shift_register_simulation();
2
3     reg clk;
4     reg clrn;
5     reg [1:0] select;
6     reg [1:0] input_rl;
7     reg [7:0] D;
8     wire [7:0] led;
9
10    always #5 clk = ~clk;
11
12    shift_register sh_r_simulate(.clrn(clrn), .sl(select[1]), .s0(select[0]), .Dsr(input_rl[1]), .Dsl(input_rl[0]),
13                                .D(D), .clk(clk), .Q(led[7:0]));
14
15    initial
16    begin
17        #0 clk = 0;
18        clrn = 1;
19        input_rl = 2'b00;
20        select = 2'b00;
21        #10
22        select = 2'b11;
23        D = 8'b11100011;
24        #10 select = 2'b11;
25        #10 select = 2'b01;
26        input_rl[1] = 0;
27        #10 clrn = 0;
28        #10 $finish;
29    end
30
31 endmodule
32

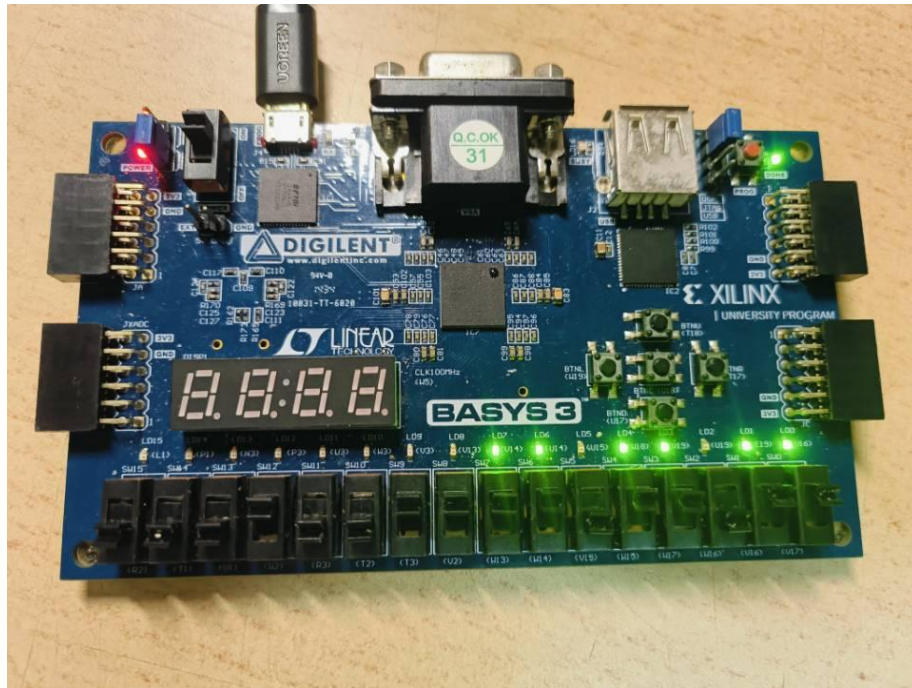
```

(3) 仿真波形图

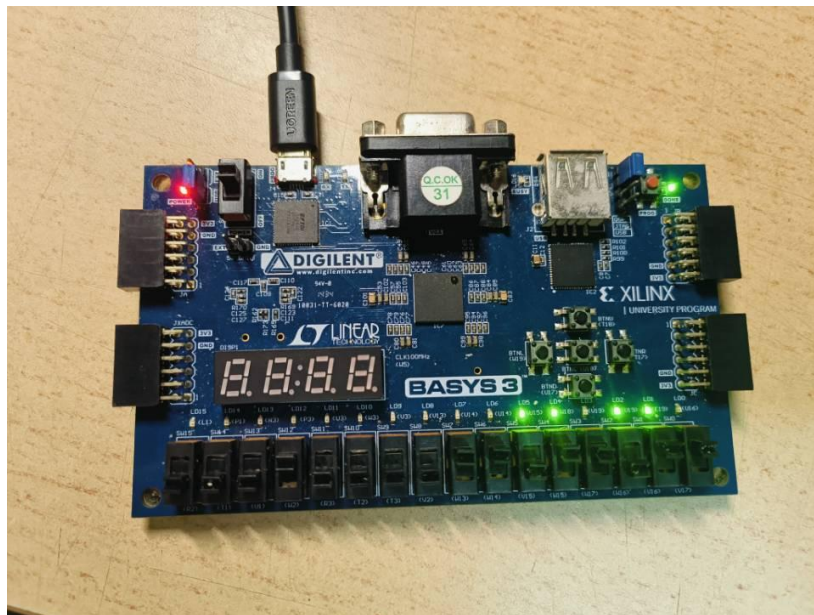


(4) 开发板测试照片

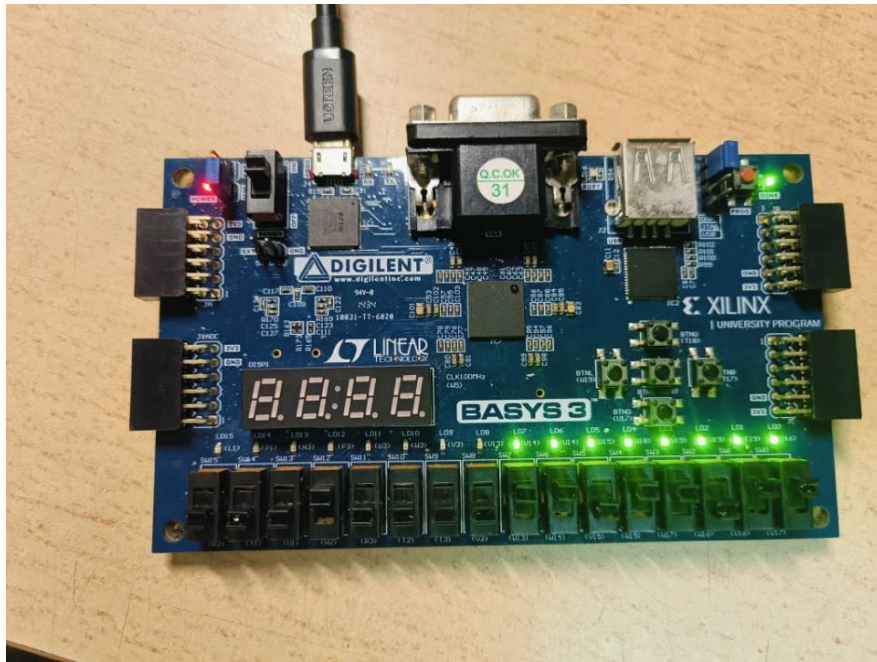
- sw[12]=1（一直保持该状态），sw[9:8]=11，并行载入 8 位数据 11011011



- $sw[11]=0$, $sw[9:8]=01$, 实现两秒的数据右移



- $sw[10]=1$, $sw[9:8]=10$, 实现数据的左移, 直到所有数据均为 1



2. Verilog 实现 4 位同步十进制计数器

(1) 主要模块代码

```
1  module clock_divide(clk, resetn, divClk);
2      input clk, resetn;
3      output reg divClk;
4
5      parameter n = 20'd1000_000;
6      reg [18:0] cnt;
7      always @(posedge clk, negedge resetn)
8          begin
9              if (!resetn)
10                 begin
11                     divClk <= 0;
12                     cnt <= 0;
13                 end
14             else
15                 begin
16                     if (cnt == n / 2 - 1)
17                         begin
18                             divClk <= ~divClk;
19                             cnt <= 0;
20                         end
21                     else
22                         cnt <= cnt + 1;
23                 end
24             end
25 endmodule
```



```
1  module dec_counter(clk, reset, enable, dec0, dec1, dec2, dec3);
2      input clk, reset, enable;
3      output reg [3:0] dec0, dec1, dec2, dec3;
4
5      always @(posedge clk)
6      begin
7          if (reset)
8              begin
9                  dec0 <= 4'b0000;
10                 dec1 <= 4'b0000;
11                 dec2 <= 4'b0000;
12                 dec3 <= 4'b0000;
13             end
14         else
15             begin
16                 if (enable)
17                     begin
18                         if (dec0 <= 4'b1000) //判断0号位
19                             dec0 <= dec0 + 1;
20                         else
21                             begin
22                                 dec0 <= 0;
23                                 if (dec1 <= 4'b1000) //判断1号位
24                                     dec1 <= dec1 + 1;
25                                 else
26                                     begin
27                                         dec1 <= 0;
28                                         if (dec2 <= 4'b1000) //判断2号位
29                                             dec2 <= dec2 + 1;
30                                         else
31                                             begin
32                                                 dec2 <= 0;
33                                                 if (dec3 <= 4'b1000) //判断3号位
34                                                     dec3 <= dec3 + 1;
35                                                 else
36                                                     begin
37                                                         dec3 <= 0;
38                                                         end
39                                                     end
40                                                 end
41                                             end
42                                         end
43                                     end
44                                 end
45                             end
46                         end
47                     end
48                 end
49             end
50         end
51     end
52 endmodule
```

```

1  module bcd_decoder(clk, dec0, dec1, dec2, dec3, an, seg);
2      input clk;
3      input [3:0] dec0, dec1, dec2, dec3;
4      output reg [3:0] an;
5      output reg [0:6] seg;
6
7      reg [16:0] regN;
8      reg [3:0] dec;
9
10     always @(posedge clk)
11         regN <= regN + 1;
12
13     always @(*)
14     begin
15         case (regN[16:15])
16             2'b00:
17                 begin
18                     dec = dec0;
19                     an = 4'b1110;
20                 end
21             2'b01:
22                 begin
23                     dec = dec1;
24                     an = 4'b1101;
25                 end
26             2'b10:
27                 begin
28                     dec = dec2;
29                     an = 4'b1011;
30                 end
31             default:
32                 begin
33                     dec = dec3;
34                     an = 4'b0111;
35                 end
36         endcase
37     end
38
39     always @(*)
40     begin
41         case (dec)
42             4'b0000: seg = 7'b0000001;
43             4'b0001: seg = 7'b1001111;
44             4'b0010: seg = 7'b0010010;
45             4'b0011: seg = 7'b0000110;
46             4'b0100: seg = 7'b1001100;
47             4'b0101: seg = 7'b0100100;
48             4'b0110: seg = 7'b0100000;
49             4'b0111: seg = 7'b0001111;
50             4'b1000: seg = 7'b0000000;
51             4'b1001: seg = 7'b0000100;
52         endcase
53     end
54
55 endmodule

```

```

1 module dec_counter_display_top(clk, sw, an, seg);
2     input clk;
3     input [1:0] sw;
4     output [3:0] an;
5     output [0:6] seg;
6
7     wire divClk;
8     clock_divide clk_div(.clk(clk), .resetn(1), .divClk(divClk));
9
10    wire [3:0] dec0, dec1, dec2, dec3;
11    dec_counter dec_cnter(.clk(divClk), .reset(sw[0]), .enable(sw[1]),
12                          .dec0(dec0), .dec1(dec1), .dec2(dec2), .dec3(dec3));
13
14    bcd_decoder bcd_dcdr(.clk(clk), .dec0(dec0), .dec1(dec1), .dec2(dec2), .dec3(dec3),
15                        .an(an[3:0]), .seg(seg[0:6]));
16 endmodule
17

```

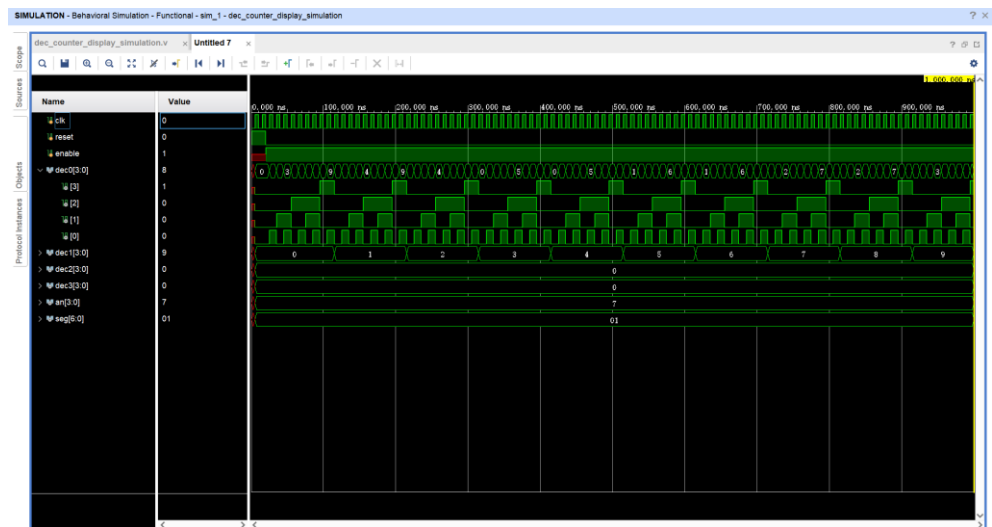
(2) 仿真测试代码

```

1 module dec_counter_display_simulation();
2     reg clk, reset, enable;
3     wire [3:0] dec0, dec1, dec2, dec3;
4     wire [3:0] an;
5     wire [6:0] seg;
6
7     always #5 clk = ~clk;
8     dec_counter dec_cnter(.clk(clk), .reset(reset), .enable(enable),
9                          .dec0(dec0), .dec1(dec1), .dec2(dec2), .dec3(dec3));
10
11    bcd_decoder bcd_dcdr(.clk(clk), .dec0(dec0), .dec1(dec1), .dec2(dec2), .dec3(dec3),
12                        .an(an[3:0]), .seg(seg[6:0]));
13
14    initial begin
15        #0 clk = 0;
16        reset = 1;
17
18        #20 reset = 0;
19        enable = 1;
20
21        #2000 $finish;
22    end
23 endmodule
24

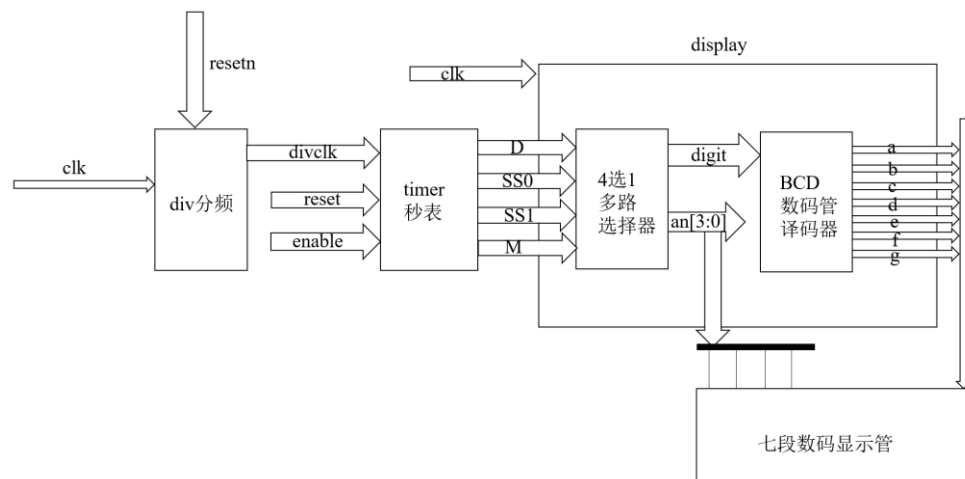
```

(3) 仿真波形图



3. Verilog 实现秒表

(1) 绘制主要块连接的逻辑框图



(2) 主要模块代码

```
1 //make the 10Hz(period 0.1s) divClk
2 module div(clk, resetn, divClk);
3     input clk, resetn;
4     output reg divClk;
5
6     parameter n = 24'd10_000_000;
7     reg [22:0] cnt;
8
9     always @(posedge clk, negedge resetn)
10    begin
11        if (!resetn)
12            divClk <= 0;
13        else
14            begin
15                if (cnt == n / 2 - 1)
16                    begin
17                        divClk <= ~divClk;
18                        cnt <= 0;
19                    end
20                else
21                    cnt <= cnt + 1;
22            end
23    end
24 endmodule
```



```
1 module timer(clk, reset, enable, M, SS0, SS1, D);
2     input clk, reset, enable;
3     output reg [3:0] D;
4     output reg [3:0] SS0;
5     output reg [3:0] SS1;
6     output reg [3:0] M;
7
8     always @(posedge clk)
9     begin
10         if (reset)
11         begin
12             D <= 4'b0000;
13             SS0 <= 4'b0000;
14             SS1 <= 4'b0000;
15             M <= 4'b0000;
16         end
17         else
18         begin
19             if (enable)
20             begin
21                 if (D <= 4'b1000) //处理D
22                     D <= D + 1;
23                 else
24                 begin
25                     D <= 0;
26                     if (SS0 <= 4'b1000) //处理SS0
27                         SS0 <= SS0 + 1;
28                     else
29                     begin
30                         SS0 <= 0;
31                         if (SS1 <= 4'b0100) //处理SS1
32                             SS1 <= SS1 + 1;
33                         else
34                         begin
35                             SS1 <= 0;
36                             if (M <= 4'b1000) //处理M
37                                 M <= M + 1;
38                             else
39                                 M <= 0;
40                         end
41                     end
42                 end
43             end
44         end
45     end
46 endmodule
```

```

1  module display(clk, M, SS1, SS0, D, an, seg);
2      input clk;
3      input [3:0] M, SS1, SS0, D;
4      output reg [3:0] an;
5      output reg [0:6] seg;
6
7      reg [18:0] regN;
8      reg [3:0] digit;
9
10     always @(posedge clk)
11         regN <= regN + 1;
12
13     always @(*)
14     begin
15         case (regN[18:17])
16             2'b00:
17                 begin
18                     digit = D;
19                     an = 4'b1110;
20                 end
21
22             2'b01:
23                 begin
24                     digit = SS0;
25                     an = 4'b1101;
26                 end
27
28             2'b10:
29                 begin
30                     digit = SS1;
31                     an = 4'b1011;
32                 end
33             default:
34                 begin
35                     digit = M;
36                     an = 4'b0111;
37                 end
38         endcase
39     end
40
41     always @(*)
42     begin
43         case (digit)
44             4'b0000: seg = 7'b0000001;
45             4'b0001: seg = 7'b1001111;
46             4'b0010: seg = 7'b0010010;
47             4'b0011: seg = 7'b0000110;
48             4'b0100: seg = 7'b1001100;
49             4'b0101: seg = 7'b0100100;
50             4'b0110: seg = 7'b0100000;
51             4'b0111: seg = 7'b0001111;
52             4'b1000: seg = 7'b0000000;
53             4'b1001: seg = 7'b0000100;
54         endcase
55     end
56
57 endmodule

```

```

1 module stop_watch_top(clk, sw, an, seg);
2     input clk;
3     input [1:0] sw;
4     output [3:0] an;
5     output [0:6] seg;
6
7     wire divClk;
8     div dv(.clk(clk), .resetn(1), .divClk(divClk));
9
10    wire [3:0] M, SS1, SS0, D;
11    timer tmr(.clk(divClk), .reset(sw[0]), .enable(sw[1]),
12             .M(M[3:0]), .SS1(SS1[3:0]), .SS0(SS0[3:0]), .D(D[3:0]));
13
14    display dpy(.clk(clk), .M(M[3:0]), .SS1(SS1[3:0]), .SS0(SS0[3:0]), .D(D[3:0]),
15              .an(an[3:0]), .seg(seg[0:6]));
16
17 endmodule

```

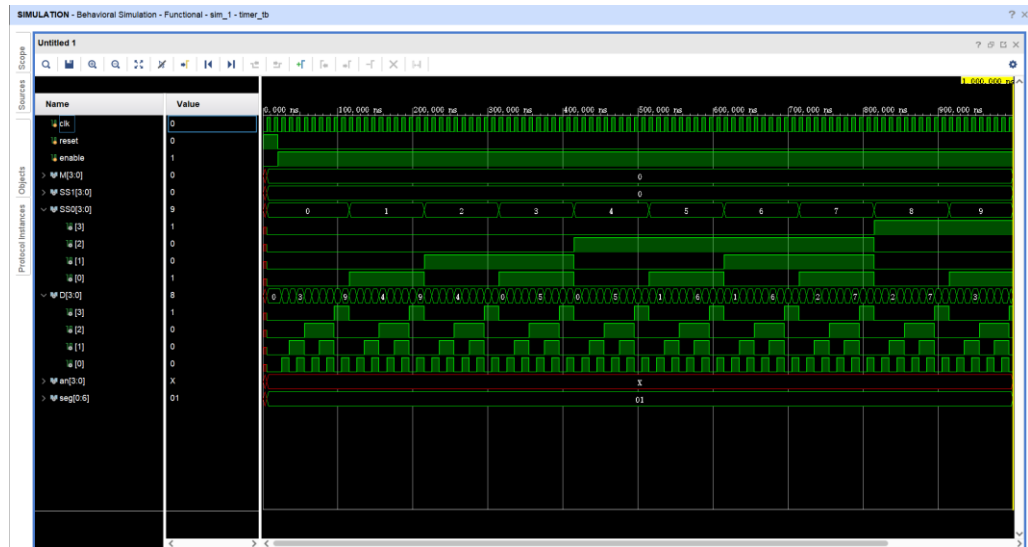
(3) 仿真测试代码

```

1 module timer_tb();
2     reg clk, reset, enable;
3     wire [3:0] M, SS1, SS0, D;
4     wire [3:0] an;
5     wire [0:6] seg;
6
7     always #5 clk = ~clk;
8
9     timer tmr(.clk(clk), .reset(reset), .enable(enable),
10             .M(M), .SS1(SS1), .SS0(SS0), .D(D));
11    display dspy(.clk(clk), .M(M), .SS1(SS1), .SS0(SS0), .D(D),
12              .an(an[3:0]), .seg(seg[0:6]));
13
14    initial begin
15        #0 clk = 0;
16        reset = 1;
17        enable = 0;
18        #20 reset = 0;
19        enable = 1;
20        #2000 $finish;
21    end
22 endmodule

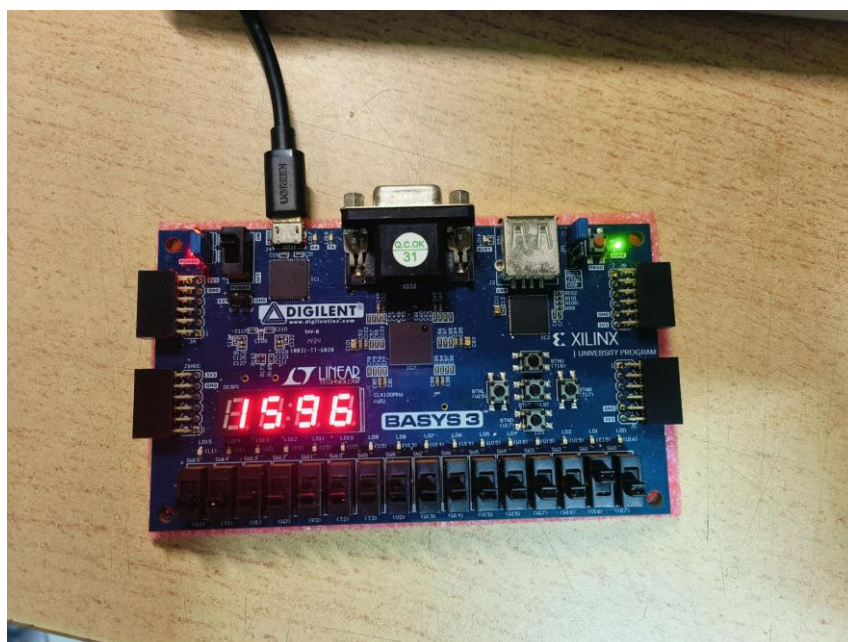
```

(4) 仿真波形图



(5) 开发板测试照片

- 计时 1m59.6s 的测试照片



附格式要求说明：

使用 Office 撰写（学校有正版授权）避免格式混乱。

正文要求：中文宋体，英文 Times New Roman，小四，1.5 倍行距，首行缩进 2 字符，段前段后 0 行。

版面要求：不要大量留白，调整图片大小及位置。

表的要求，题注在表上方居中，宋体，五号字，加黑，按一级标题编号，文本单倍行距。

逻辑框图绘制使用 Visio 或这 PowerPoint 绘制，然后全选复制粘贴到 Word 中。

图标题要求：宋体，五号字，加黑，按一级标题编号。

代码要求：字体 Consolas，五号字，1 倍行距；也可以截图展示。

```
module mux (  
    input sel,  
    input d0,  
    input d1,  
    output y);  
  
    assign y = sel?d1:d0;  
endmodule
```